

Lab 3 Report

Summer 2026

Soeun Jang
Virginia Tech
`sophiaj04@vt.edu`

June 15, 2026

Contents

Introduction

Part 1: Tensor Basics

Exercise 1

1. Unfold function

```
function unfold(A::Array{Float64,3}, n::Int)::Matrix{Float64}
    n1, n2, n3 = size(A)
    if n == 1
        return reshape(permutedims(A, [1,2,3]), n1, n2*n3)
    elseif n == 2
        return reshape(permutedims(A, [2,1,3]), n2, n1*n3)
    else
        return reshape(permutedims(A, [3,1,2]), n3, n1*n2)
    end
end
```

Based on the function given, we implemented unfold function that returns those values. To verify this function works,

```
function unfold(A::Array{Float64,3}, n::Int)::Matrix{Float64}
    n1, n2, n3 = size(A)
    if n == 1
        return reshape(permutedims(A, [1,2,3]), n1, n2*n3)
    elseif n == 2
        return reshape(permutedims(A, [2,1,3]), n2, n1*n3)
    else
        return reshape(permutedims(A, [3,1,2]), n3, n1*n2)
    end
end
```

We used this verification and every assertion passed which confirms that the implemented unfold function successfully rearranges the tensor structure into appropriate 2-dimensional matrix form for each mode.

2. Implement mode product function

```
function mode_product(A::Array{Float64,3}, M::Matrix{Float64}, n::Int)::Array{Float64,3}
    n1, n2, n3 = size(A)
    s = size(M, 1)

    An = unfold(A, n)
    B_unfolded = M * An

    if n == 1
        return permutedims(reshape(B_unfolded, s, n2, n3), [1,2,3])
    elseif n == 2
        return permutedims(reshape(B_unfolded, s, n1, n3), [2,1,3])
    else
        return permutedims(reshape(B_unfolded, s, n1, n2), [2,3,1])
    end
end
```

The mode n product between a 3 way tensor and a matrix, those are defined as multiplying M with the mode unfolding of A then just folding the result back into the size tensor. To verify this function,

```
#Ex 1-2
A = rand(4, 5, 6)
M1 = rand(3, 4)
B = mode_product(A, M1, 1)
println("mode_product(A, M1, 1) size: ", size(B))
@assert size(B) == (3, 5, 6)
println("passed for mode_product with n=1.")
```

This prints out "mode_product(A, M1, 1) size: (3, 5, 6) passed for mode_product with n=1." which indicates that it passes. The resulting tensor has a size of (3,5,6) which means that it confirms the implementation is correct since multiplication of mode-1 changes into 3.

3. Verification of Commutativity error

```
#Ex 1-3
M2 = rand(7, 5)

C1 = mode_product(mode_product(A, M1, 1), M2, 2)
C2 = mode_product(mode_product(A, M2, 2), M1, 1)

err = maximum(abs.(C1 - C2))
println("commutativity error: ", err)
@assert err < 1e-12
println("passed for commutativity of mode_product.")
```

Based on same A, M1, commutativity error: 8.881784197001252e-16 passed for commutativity of mode_product. Since the multiplication of mode-1 and mode-2 are commuting each other which means they are always identical tensor.

4. Separable tensor verification

```
n = 20
pts = range(0, pi, length=n)

f1 = sin.(pts)
f2 = cos.(2 .* pts)
f3 = exp.(.-pts)

# A[i,j,k] = f1(xi) * f2(yj) * f3(zk)
A_sep = [f1[i] * f2[j] * f3[k] for i in 1:n, j in 1:n, k in 1:n]

A1 = unfold(A_sep, 1)
A2 = unfold(A_sep, 2)
A3 = unfold(A_sep, 3)

sv1 = svdvals(A1)
sv2 = svdvals(A2)
sv3 = svdvals(A3)

tol = 1e-10
r1 = sum(sv1 .> tol)
r2 = sum(sv2 .> tol)
r3 = sum(sv3 .> tol)

println("rank of mode-1 unfolding: ", r1)
println("rank of mode-2 unfolding: ", r2)
println("rank of mode-3 unfolding: ", r3)
```

This prints out rank of mode-1 unfolding: 1 rank of mode-2 unfolding: 1 rank of mode-3 unfolding: 1 since every separated tensor $A_{ijk} = f_1(x_i)f_2(yIj)f_3(z_k)$ contains three vector's external structure, three mode unfolding rank should be all 1.

Part 2: HOSVD and the Tucker Struct

Exercise 2

1. Define the Tucker data structure

```
mutable struct Tucker3
    G::Array{Float64,3} # r1 x r2 x r3 core tensor
    U1::Matrix{Float64} # n1 x r1
    U2::Matrix{Float64} # n2 x r2
    U3::Matrix{Float64} # n3 x r3
end
```

This tucker3 structure is the data structure to store tucker decomposition by G which is small core tensor and U1, U2, U3 works as a base that is the factor matrices of the mode. Therefore, this structure captures both the compressed core and the mode wise basis matrices that reconstruct the full tensor.

2. Implement todense

```
function todense(T::Tucker3)::Array{Float64,3}
    A = mode_product(T.G, T.U1, 1)
    A = mode_product(A, T.U2, 2)
    A = mode_product(A, T.U3, 3)
    return A
end
```

The todense function reconstructs the full tensor by applying the tree mode products and it expands the compressed core G tensor back to its original dimensions.

3. Implement hosvd function

```
function hosvd(A::Array{Float64,3}, r::Tuple{Int,Int,Int})::Tucker3
    r1, r2, r3 = r

    U1 = svd(unfold(A, 1)).U[:, 1:r1]
    U2 = svd(unfold(A, 2)).U[:, 1:r2]
    U3 = svd(unfold(A, 3)).U[:, 1:r3]

    G = mode_product(A, U1', 1)
    G = mode_product(G, U2', 2)
    G = mode_product(G, U3', 3)

    return Tucker3(G, U1, U2, U3)
end
```

In the HOSVD procedure, we first compute the mode-1, mode-2, and mode-3 unfoldings of the tensor A. From each unfolding, we apply SVD and take the leading r1, r2, r3 to form the factor matrices U1, U2, U3 each. After obtaining these factor matrices, we compute the core tensor G by multiplying the original tensor A with the transposes of the factor matrices along each side and this core tensor G with U1, U2, U3 forms the tucker3 decomposition.

4. Implement Tucker3 function

```

function Tucker3(A::Array{Float64,3}, tol::Float64)::Tucker3
    threshold = tol / sqrt(3) * norm(A)

    F1 = svd(unfold(A, 1))
    F2 = svd(unfold(A, 2))
    F3 = svd(unfold(A, 3))

    r1 = max(1, sum(F1.S .>= threshold))
    r2 = max(1, sum(F2.S .>= threshold))
    r3 = max(1, sum(F3.S .>= threshold))

    U1 = F1.U[:, 1:r1]
    U2 = F2.U[:, 1:r2]
    U3 = F3.U[:, 1:r3]

    # Core tensor
    G = mode_product(A, Matrix(U1'), 1)
    G = mode_product(G, Matrix(U2'), 2)
    G = mode_product(G, Matrix(U3'), 3)

    return Tucker3(G, U1, U2, U3)
end

```

This function makes tensor A be compressed as the way of tucker. Based on the tolerance called tol , it chooses the multi-linear ranks r value that is greater than threshold. Then, it applies SVD for each mode unfolding A to get the factor matrices $U1$, $U2$, $U3$ that correspond to each directions of modes 1,2,3. To get core tensor G by compressing each directions and return the structure with tucker3 structure.

5. Verification

```

include("../src/unfold.jl")
include("../src/mode.product.jl")
include("../src/tucker3.jl")
include("../src/tucker_3.jl")
include("../src/todense.jl")
include("../src/hosvd.jl")

using LinearAlgebra

A = randn(30, 30, 30)
T = Tucker3(A, 1e-12)

err = norm(todense(T) - A)
println("norm of A - todense(Tucker3(A, 1e-12))= ", err)
println("1e-10 * norm(A)= ", 1e-10 * norm(A))
@assert err < 1e-10 * norm(A)
println("Verified that the error is within the specified tolerance.")

```

Based on this verification, it prints out "norm of A - todense(Tucker3(A, 1e-12))= 3.8296095413294746e-13" and "1e-10 * norm(A)= 1.6477528417710177e-8". It verifies that the error is within the specified tolerance. In conclusion, tucker3 decomposition works correctly.

Part 3: Tucker Truncation and Error Analysis

Exercise 3

1. Implement tucker round function

```
function tucker_round(T::Tucker3, tol::Float64)::Tucker3
    G = T.G
    U1 = T.U1
    U2 = T.U2
    U3 = T.U3

    F1 = svd(unfold(G, 1))
    F2 = svd(unfold(G, 2))
    F3 = svd(unfold(G, 3))

    threshold = tol / sqrt(3) * norm(G)

    r1 = max(1, sum(F1.S .>= threshold))
    r2 = max(1, sum(F2.S .>= threshold))
    r3 = max(1, sum(F3.S .>= threshold))

    Ub1 = F1.U[:, 1:r1]
    Ub2 = F2.U[:, 1:r2]
    Ub3 = F3.U[:, 1:r3]

    Gnew = mode_product(G, Matrix(Ub1'), 1)
    Gnew = mode_product(Gnew, Matrix(Ub2'), 2)
    Gnew = mode_product(Gnew, Matrix(Ub3'), 3)

    U1new = U1 * Ub1
    U2new = U2 * Ub2
    U3new = U3 * Ub3

    return Tucker3(Gnew, U1new, U2new, U3new)
end
```

This function explains the tucker round which applies HOSVD to the core G , then update the factor matrices without ever expanding to the full tensor.

2. Test

```
xvinclude("../src/unfold.jl")
include("../src/mode.product.jl")
include("../src/tucker3.jl")
include("../src/tucker_3.jl")
include("../src/todense.jl")
include("../src/hosvd.jl")
include("../src/tucker_round.jl")

using Test
using LinearAlgebra

@testset "Tucker_Round Test" begin
    n1 = n2 = n3 = 30
    r1 = r2 = r3 = 20

    U1 = randn(n1, r1)
    U2 = randn(n2, r2)
```

```

U3 = randn(n3, r3)
G = randn(r1, r2, r3)

A = mode_product(mode_product(mode_product(G, U1, 1), U2, 2), U3, 3)

T = Tucker3(A, 1e-12)

tols = [1e-2, 1e-4, 1e-8]

for tol in tols
    Tr = tucker_round(T, tol)

    r = (size(Tr.G,1), size(Tr.G,2), size(Tr.G,3))

    err = norm(todense(Tr) - A) / norm(A)

    @test err < 10 * tol
    @test all(ri <= 20 for ri in r)

    println("tol = $tol ranks = $r rel error = $err")
end
end

```

This test gives the error value about $3.41e - 15$ since we are rounding the core which is already orthogonally compressed for every tolerance and rank (20, 20, 20). Since the synthetic tensor actually has true multi-linear rank, all tolerances give rank of 20. Also the singular values of the core are all large, and non fall below $10^{-2}, 10^{-4}, 10^{-8}$ which means there is no truncation so the rounded ranks stay at 20, 20, 20.

3. HOSVD multi-linear rank

```

@testset "Gaussian Tucker Rank Test" begin
    n = 100
    xs = range(-3, 3, length=n)
    ys = range(-3, 3, length=n)
    zs = range(-3, 3, length=n)

    A = Array{Float64,3}(undef, n, n, n)
    for i in 1:n, j in 1:n, k in 1:n
        A[i,j,k] = exp(-(xs[i]^2 + ys[j]^2 + zs[k]^2))
    end

    T = Tucker3(A, 1e-12)

    r = (size(T.G,1), size(T.G,2), size(T.G,3))
    println("Multilinear rank assigned by HOSVD = ", r)

    @test r[1] >= 1
    @test r[2] >= 1
    @test r[3] >= 1

    err = norm(todense(T) - A) / norm(A)
    println("Error = ", err)

    @test err < 1e-10
end

```

Confirming 3D Gaussian Tucker rank, this returns the multilinear rank (1,1,1) with error about

1.12e-14. HOSVD detects the correct rank. Also the error is tiny, which is an almost exact representation of the discrete tensor.

Exercise 4

```
using Test
using LinearAlgebra
using Plots

# helper function to unfold a 3D array along mode n
function unfold(A, n)
    if n == 1
        return reshape(A, size(A,1), :)
    elseif n == 2
        return reshape(permutedims(A, (2,1,3)), size(A,2), :)
    else
        return reshape(permutedims(A, (3,1,2)), size(A,3), :)
    end
end

# singular values of mode-n unfolding
function mode_svals(A)
    s1 = svd(unfold(A,1)).S
    s2 = svd(unfold(A,2)).S
    s3 = svd(unfold(A,3)).S
    return s1, s2, s3
end

@testset "Exercise 4: Singular Value Decay" begin
    n = 50

    #1
    xs = range(0, 2, length=n)
    ys = range(0, 2, length=n)
    zs = range(0, 2, length=n)

    A1 = [sin(x+y+z) for x in xs, y in ys, z in zs]
    s1a, s2a, s3a = mode_svals(A1)

    r1a = sum(s1a .> 1e-6)
    r2a = sum(s2a .> 1e-6)
    r3a = sum(s3a .> 1e-6)

    println("\n(1) sin(x+y+z) ranks = ", (r1a, r2a, r3a))

    #2
    eps = 0.1
    xs = range(0, 1, length=n)
    ys = range(0, 1, length=n)
    zs = range(0, 1, length=n)

    A2 = [1/sqrt(x^2 + y^2 + z^2 + eps^2) for x in xs, y in ys, z in zs]
    s1b, s2b, s3b = mode_svals(A2)

    r1b = sum(s1b .> 1e-6)
    r2b = sum(s2b .> 1e-6)
    r3b = sum(s3b .> 1e-6)
end
```

```

println("(2) 1/sqrt(x^2+y^2+z^2+eps^2) ranks = ", (r1b, r2b, r3b))

#3
xs = range(-1, 1, length=n)
ys = range(-1, 1, length=n)
zs = range(-1, 1, length=n)

A3 = [tanh(10*(x^2 + y^2 - z)) for x in xs, y in ys, z in zs]
s1c, s2c, s3c = mode_svals(A3)

r1c = sum(s1c .> 1e-6)
r2c = sum(s2c .> 1e-6)
r3c = sum(s3c .> 1e-6)

println("(3) tanh(10(x^2+y^2-z)) ranks = ", (r1c, r2c, r3c))

#plotting singular value decay
p1 = plot(s1a, ylabel=:log10, label="mode-1", title="sin(x+y+z)")
plot!(p1, s2a, label="mode-2")
plot!(p1, s3a, label="mode-3")
savefig(p1, "p1.png")
display(p1)

p2 = plot(s1b, ylabel=:log10, label="mode-1", title="1/sqrt(x^2+y^2+z^2+eps^2)")
plot!(p2, s2b, label="mode-2")
plot!(p2, s3b, label="mode-3")
savefig(p2, "p2.png")
display(p2)

p3 = plot(s1c, ylabel=:log10, label="mode-1", title="tanh(10(x^2+y^2-z))")
plot!(p3, s2c, label="mode-2")
plot!(p3, s3c, label="mode-3")

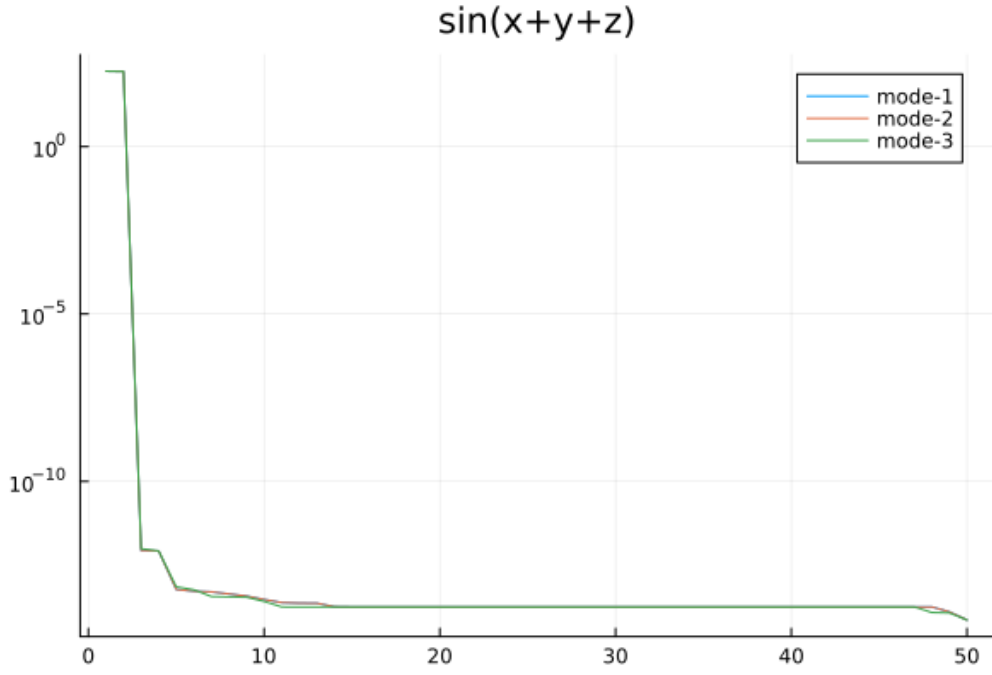
savefig(p3, "p3.png")
display(p3)

plot(p1, p2, p3, layout=(3,1), size=(800,1200))
end

```

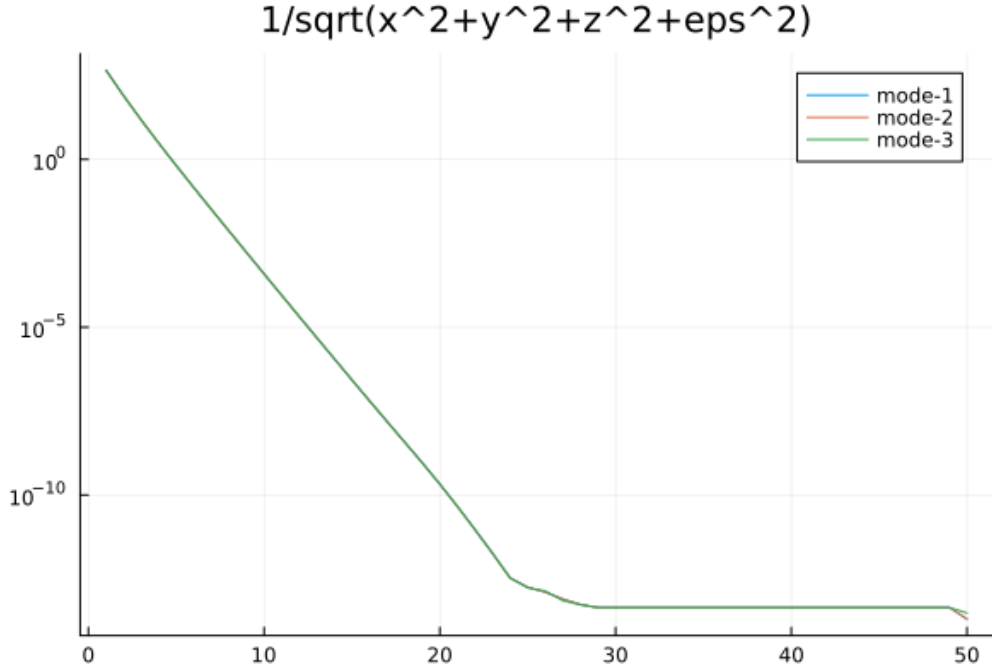
1. $\sin(x + y + z)$

The ranks are (2, 2, 2) which is symmetric balanced. Each mode unfolding sees the same structure and the singular values decay identically across modes. This is the most balanced case.



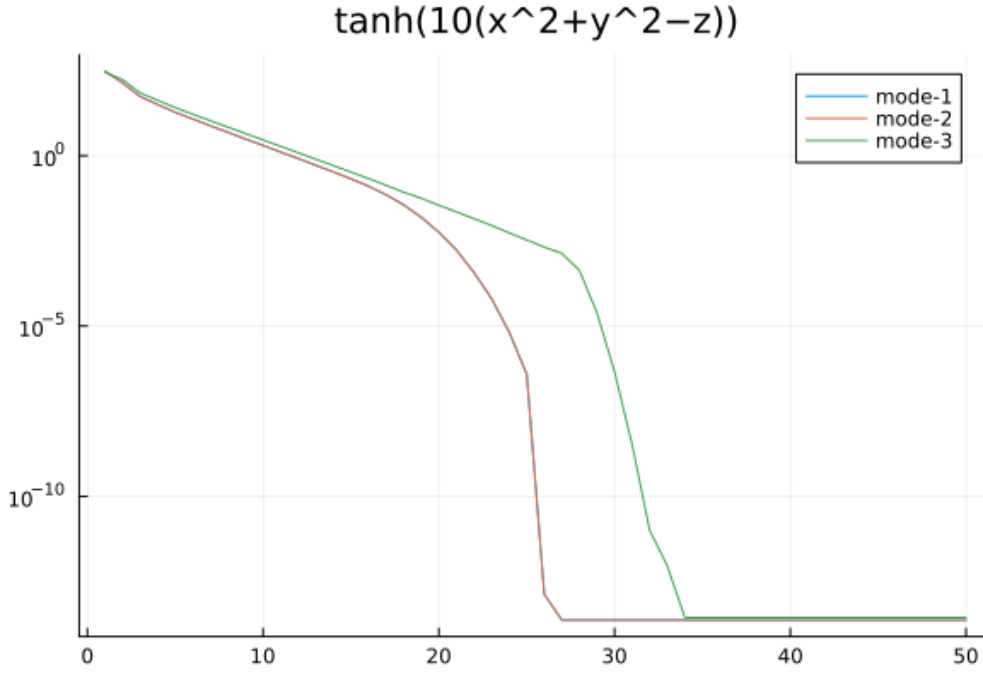
2. $1/\sqrt{x^2 + y^2 + z^2 + \epsilon^2}$

The ranks are (14, 14, 14) which is balanced but it contains higher rank. This function is radially symmetric which means the function depends only on the distance from the origin.



3. $\tanh(10(x^2 + y^2 - z))$

The ranks are (24, 24, 29) which is unbalanced. This function treats the variables asymmetrically. x, y shows quadratically and symmetrically but z appears linearly and contains opposite sign.



Part 4: Tucker Arithmetic and 3D PDE Setup

Exercise 5

1. Inner product of two tucker tensors

Let

$$A = G \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)}$$

and

$$B = H \times_1 W^{(1)} \times_2 W^{(2)} \times_3 W^{(3)}$$

G, H are core tensor and $U^{(n)}, W^{(n)}$ are factor matrices and $\times_n$ is mode- n product. Frobenius of those two tensors is

$$(A, B)_F = \sum_{i_1, i_2, i_3} A_{i_1, i_2, i_3} B_{i_1, i_2, i_3}$$

when we apply tucker structure, since

$$A_{i_1, i_2, i_3} = \sum_{j_1, j_2, j_3} G_{j_1, j_2, j_3} U_{i_1, j_1}^{(1)} U_{i_2, j_2}^{(2)} U_{i_3, j_3}^{(3)}$$

$$B_{i_1, i_2, i_3} = \sum_{k_1, k_2, k_3} H_{k_1, k_2, k_3} W_{i_1, k_1}^{(1)} W_{i_2, k_2}^{(2)} W_{i_3, k_3}^{(3)}$$

the frobenius formula definition is equal to

$$(A, B)_F = \sum_{j_1, j_2, j_3} \sum_{k_1, k_2, k_3} G_{j_1, j_2, j_3} H_{k_1, k_2, k_3} \left(U^{(1)} \right)^T W_{j_1, k_1}^{(1)} \left(U^{(2)} \right)^T W_{j_2, k_2}^{(2)} \left(U^{(3)} \right)^T W_{j_3, k_3}^{(3)}$$

that is the final inner formula based on multiplication of each core tensors and factor matrices of each mode. This is effective since its computational cost is lower than dense tensor which indicates that the computational speed is faster than normal.

2. Implement inner tucker

```

function tucker_inner(A::Tucker3, B::Tucker3)::Float64
    M1 = A.U1' * B.U1
    M2 = A.U2' * B.U2
    M3 = A.U3' * B.U3
    H_contracted = mode_product(B.G, M1', 1)
    H_contracted = mode_product(H_contracted, M2', 2)
    H_contracted = mode_product(H_contracted, M3', 3)

    return sum(A.G .* H_contracted)
end

```

Based on the code, it calculates the multiplication of each tensors.

To verify,

```

include("../src/unfold.jl")
include("../src/mode_product.jl")
include("../src/tucker3.jl")
include("../src/tucker_3.jl")
include("../src/todense.jl")
include("../src/tucker_inner.jl")
using LinearAlgebra

```

```

A_dense = randn(10, 10, 10)
B_dense = randn(10, 10, 10)

```

```

TA = Tucker3(A_dense, 1e-14)
TB = Tucker3(B_dense, 1e-14)

```

```

ip_tucker = tucker_inner(TA, TB)
ip_dense = sum(todense(TA) .* todense(TB))
ip_exact = sum(A_dense .* B_dense)

println("tucker_inner: ", ip_tucker)
println("dense inner: ", ip_dense)
println("exact inner: ", ip_exact)
println("error vs exact: ", abs(ip_tucker - ip_exact))
@assert abs(ip_tucker - ip_exact) < 1e-10
println("tucker_inner passed")

```

It prints out tucker_inner: 17.17314068389137 dense inner: 17.173140683891333 exact inner: 17.173140683891194 error vs exact: 1.7408297026122455e-13 tucker_inner passed. It means that tiny error is just floating point rounding and the structure is all correct.

3. Implement tucker norm function

```

function norm_tucker(A::Tucker3)::Float64
    return sqrt(tucker_inner(A, A))
end

```

Norm is a square root of inner tucker.

Exercise 6

1. Show the tucker structure of $A + B$

With ranks r_1^A, r_2^A, r_3^A and r_1^B, r_2^B, r_3^B , the sum is all basis vectors from A and B. Therefore, the factor matrices for the sum should have all columns from both tensors.

$$U^{sum,(n)} = [U^{A,(n)} U^{B,(n)}]$$

which is equal to

$$r_1^A + r_1^B, r_2^A + r_2^B, r_3^A + r_3^B$$

that is the new multilinear rank.

The new core tensor of $A+B$ is $U^{(n)} = [U^{A,(n)}U^{B,(n)}]$ and core tensor G_A, G_B each that is a block-diagonal tensor.

$$G_{sum} = \begin{bmatrix} G_A & 0 \\ 0 & G_B \end{bmatrix}$$

since G_A multiplies the column of U^A and G_B multiplies the column of U^B in separate blocks.

2. Implement tucker add function

```
function tucker_add(A::Tucker3, B::Tucker3)::Tucker3
    U1_new = hcat(A.U1, B.U1)
    U2_new = hcat(A.U2, B.U2)
    U3_new = hcat(A.U3, B.U3)

    r1A, r2A, r3A = size(A.G)
    r1B, r2B, r3B = size(B.G)

    G_new = zeros(r1A+r1B, r2A+r2B, r3A+r3B)
    G_new[1:r1A, 1:r2A, 1:r3A] = A.G
    G_new[r1A+1:end, r2A+1:end, r3A+1:end] = B.G

    return Tucker3(G_new, U1_new, U2_new, U3_new)
end
```

This function adds tucker A and B that use its own basis vectors. $A+B$ concatenate columns and the cores do not mix and it does contain block-diagonal core.

3. Implement tucker sum function

```
function tucker_sum(terms::Vector{Tucker3}, tol::Float64)::Tucker3
    result = tucker_round(terms[1], tol)
    for i in 2:length(terms)
        result = tucker_add(result, terms[i], tol)
    end
    return result
end
```

This function sums a vector of Tucker tensors and recompresses the result with `tucker_round`.

4. Verification

```
include("../src/unfold.jl")
include("../src/mode.product.jl")
include("../src/tucker3.jl")
include("../src/tucker_3.jl")
include("../src/todense.jl")
include("../src/tucker_inner.jl")
include("../src/tucker_add.jl")
include("../src/tucker_sum.jl")

using Test
using LinearAlgebra

@testset "tucker_sum test" begin
```

```

U1 = Matrix(I, 10, 10)[: , 1:2]
U2 = Matrix(I, 10, 10)[: , 1:2]
U3 = Matrix(I, 10, 10)[: , 1:2]

Ga = ones(2,2,2)
Gb = reshape(1:8, 2,2,2)

A = Tucker3(Ga, U1, U2, U3)
B = Tucker3(Gb, U1, U2, U3)

dense_exact = todense(A) + todense(B)

Tsum = tucker_sum([A, B], 1e-12)
dense_tucker = todense(Tsum)

err = norm(dense_tucker - dense_exact) / norm(dense_exact)

@test err < 1e-10
end

```

This test contains factor matrices and cores. We get its true sum of dense tensors and it does match with the tucker sum function with error that is less than 1e-10. It verifies tucker sum function works perfectly.

Exercise 7

1. Show each term of W is Tucker

We are given that

$$\mathcal{L}(W) = D_x \times_1 W + D_y \times_2 W + D_z \times_3 W$$

and let

$$W = G \times_1 U_1 \times_2 U_2 \times_3 U_3$$

then

$$D_x \times_1 W = D_x \times_1 G \times_1 U_1 \times_2 U_2 \times_3 U_3$$

as mode-1 product with D_x , it also associates.

$$D_x \times_1 W = G \times_1 D_x U_1 \times_2 U_2 \times_3 U_3$$

which contains same core G and factor matrices-tucker structure. Since $D_x U_1$ has rank of $n \times r_1$ which is the rank at most r_1 -their rank stays same. When we apply it for every terms,

$$D_y \times_2 W = G \times_1 U_1 \times_2 D_y U_2 \times_3 U_3$$

$$D_z \times_3 W = G \times_1 U_1 \times_2 U_2 \times_3 D_z U_3$$

this is a tucker tensor with rank at most r_1, r_2, r_3 since they have same number of columns as the original factor matrices.

2. Implement applyL tucker function

```

include("tucker_sum.jl")

function applyL_tucker(W::Tucker3, Dx, Dy, Dz, tol)
    G = W.G
    U1 = W.U1
    U2 = W.U2

```

```

U3 = W.U3

T1 = Tucker3(G, Dx * U1, U2, U3)
T2 = Tucker3(G, U1, Dy * U2, U3)
T3 = Tucker3(G, U1, U2, Dz * U3)

return tucker_sum([T1, T2, T3], tol)
end

```

It uses tucker sum function to keep the rank under control and it uses the tucker structure directly.

3. Step-and-Truncate solver

```

include("../src/unfold.jl")
include("../src/mode.product.jl")
include("../src/tucker3.jl")
include("../src/tucker_3.jl")
include("../src/todense.jl")
include("../src/tucker_inner.jl")
include("../src/tucker_add.jl")
include("../src/tucker_sum.jl")
include("../src/applyL_tucker.jl")
using LinearAlgebra
# 7-3 step and truncate
n = 32
h = 2 * pi/n
x = range(0, 2*pi - h, length=n)
function D_spectral(n)
    h = 2*pi / n
    x = range(0, 2*pi - h, length=n)
    D = zeros(n, n)
    for i in 1:n, j in 1:n
        if i != j
            D[i,j] = 0.5 * (-1)^(i-j) * cot((i-j)*pi/n)
        end
    end
    return D * (2*pi / (n*h))
end
Dx = D_spectral(n)
Dy = D_spectral(n)
Dz = D_spectral(n)

function taylor_step_tucker(W, Dx, Dy, Dz, dt, p, tol)
    terms = [tucker_round(W, tol)] # W truncate
    Wk = tucker_round(W, tol)
    for k in 1:p
        Wk = applyL_tucker(Wk, Dx, Dy, Dz, tol)
        scale = dt^k / factorial(k)
        push!(terms, Tucker3(Wk.G * scale, Wk.U1, Wk.U2, Wk.U3))
    end
    return tucker_sum(terms, tol)
end

function run_step_and_truncate()
    u0_dense = [sin(xi)*sin(yj)*sin(zk)
                 for xi in x, yj in x, zk in x]

    dt = 0.1

```



```

p = 4
T = 2*pi
N_steps = round(Int, T/dt)
tol = dt^(p+1)

W = Tucker3(u0_dense, tol)
rank_history = Tuple{Int,Int,Int}[]
t0 = time()

for step in 1:N_steps
    W = taylor_step_tucker(W, Dx, Dy, Dz, dt, p, tol)
    push!(rank_history, (size(W.U1,2), size(W.U2,2), size(W.U3,2)))
    if step % 10 == 0
        elapsed = round(time()-t0, digits=1)
        println("step=$step/$N_steps, ranks=$(rank_history[end]), t=$(elapsed)s")
    end
end

println("Finished $N_steps steps")
for (step, ranks) in enumerate(rank_history)
    println("step=$step ranks=$ranks")
end

return rank_history
end

run_step_and_truncate()

```

For the 3D advection equation, I implemented the step and truncate time stepping method in tukcer format with the taylor method of order $p = 4$. Tolerance was chosen as 10^{-5} as required. The solver is extremely slow since the taylor method repeatedly applies the operator to the current tucker tensor. This is fundamentally much more expensive than RK4 step and truncate from Lab 1, which only requires four operator evaluations per step.

4. Computing error and verify convergence

```

include("../src/unfold.jl")
include("../src/mode.product.jl")
include("../src/tucker3.jl")
include("../src/tucker_3.jl")
include("../src/todense.jl")
include("../src/tucker_inner.jl")
include("../src/tucker_add.jl")
include("../src/tucker_sum.jl")
include("../src/applyL_tucker.jl")
using LinearAlgebra
using Printf

function D_spectral(n)
    h = 2*pi / n
    D = zeros(n, n)
    for i in 1:n, j in 1:n
        if i != j
            D[i,j] = 0.5 * (-1)^(i-j) * cot((i-j)*pi/n)
        end
    end
    return D * (2*pi / (n*h))
end

```

```

function taylor_step_tucker(W, Dx, Dy, Dz, dt, p, tol)
    terms = [tucker_round(W, tol)]
    Wk = tucker_round(W, tol)
    for k in 1:p
        Wk = applyL_tucker(Wk, Dx, Dy, Dz, tol)
        scale = dt^k / factorial(k)
        push!(terms, Tucker3(Wk.G * scale, Wk.U1, Wk.U2, Wk.U3))
    end
    return tucker_sum(terms, tol)
end

n = 32
h = 2*pi / n
x = range(0, 2*pi - h, length=n)
Dx = D_spectral(n)
Dy = D_spectral(n)
Dz = D_spectral(n)

u0_dense = [sin(xi)*sin(yj)*sin(zk) for xi in x, yj in x, zk in x]
u_ref = copy(u0_dense) # exact solution at T=2 equals u0

p = 4
prev_ratio = nothing

println("Convergence test (n=$n, p=$p)")
println("="^55)
println("dt | error | dt^p | ratio")
println("-"^55)

for dt in [0.2, 0.1, 0.05]
    tol = dt^(p+1)
    N_steps = round(Int, 2*pi/dt)

    W = Tucker3(u0_dense, tol)
    t0 = time()

    for step in 1:N_steps
        W = taylor_step_tucker(W, Dx, Dy, Dz, dt, p, tol)
    end

    err = h^(3/2) * norm(todense(W) - u_ref)
    ratio = err / dt^p
    elapsed = round(time()-t0, digits=1)

    @printf("dt=%.3f | err=%.4e | dt^p=%.4e | ratio=%.4f (%.1fs)\n",
           dt, err, dt^p, ratio, elapsed)

    global prev_ratio = ratio
end

```

The convergence test uses a 4th order taylor step-and-truncate method. Each steps produces three tucker tensors and each taylor term adds another tensor to the sum. This indicates that the ranks grow gradually and each truncation requires an SVD of the tucker core. In conclusion, this method is computationally expensive. Since I verified the temporal convergence rate, this method successfully implements 4th order.

Conclusion

In this lab, I implemented tucker tensor workflow for 3D data including unfolding, mode-n products, HOSVD, step and truncate, and a full 3D PDE solver. Through the exercises, we can figure out that each component behaves as expected numerically.

Part 1 shows the basic tensor operations for the accurate shapes. Part 2 implements the Tucker3 data structure that the decomposition accurately rebuilds the dense tensors with the tolerance and multi linear ranks. Part 3 describes the truncation and errors since those functions are successfully implemented without expanding to full size and its error was explained with the tolerance. It also illustrates how different functions produce balanced or not in the singular value decay experiments. In part 4, all function combined multiple tucker tensors while controlling rank growth through truncation so we applied those tools to the 3D advection. It successfully evolved the solution while keeping ranks small.

Overall, this lab showed how tucker decompositions enable efficient representation, manipulation and time difference of high dimensional data. It confirms that tucker format and its practical effectiveness for large scale PDE problems.